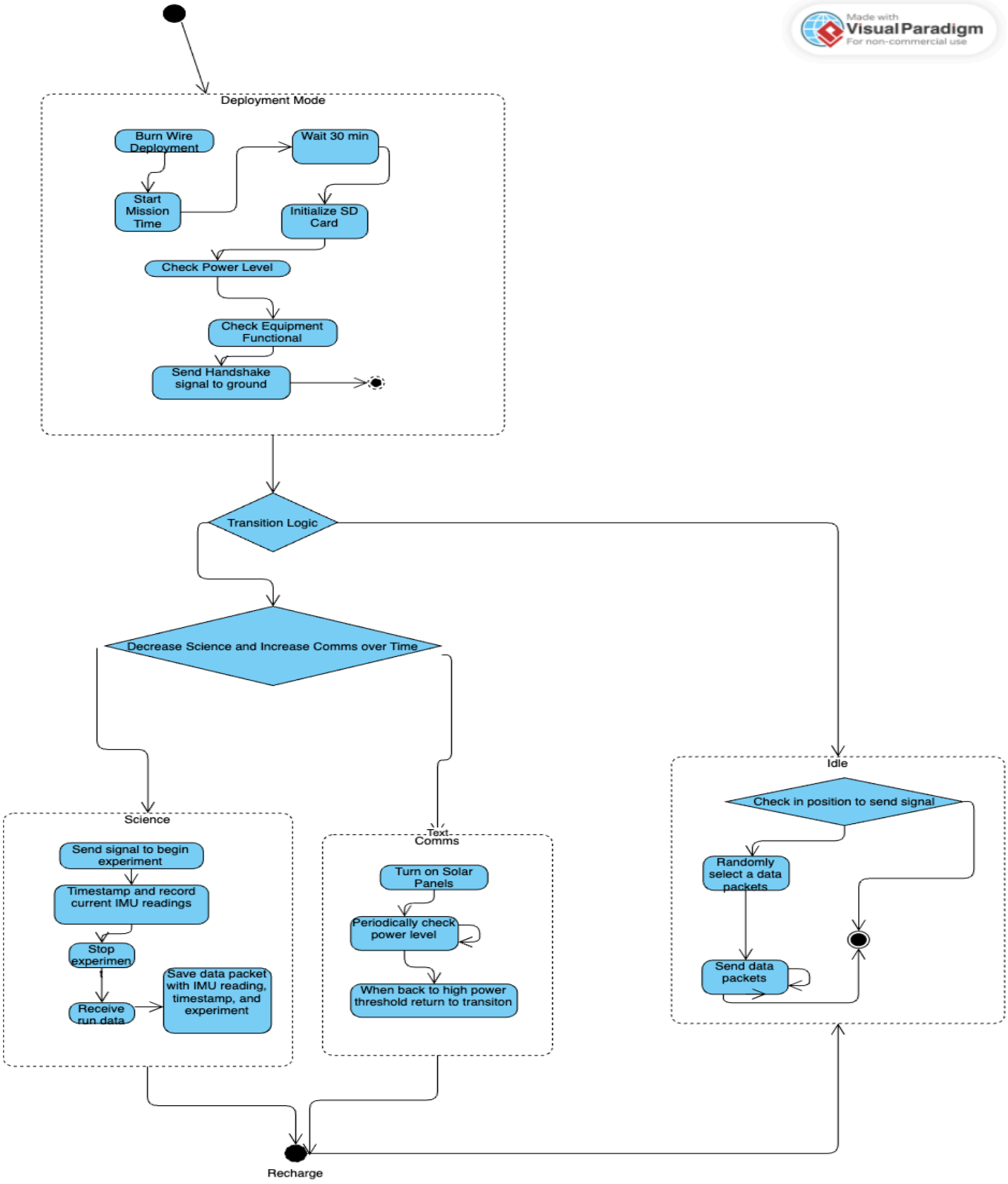
Qubesat FSM States and Tasks



Idle State Tasks

The *Idle* state is entered on reboot and in cases when cubesat power is low. Minimal [power-intensive actions are taken in the *Idle* state

Beacon
Beacon i

- Priority: 1
- Period: 30 minutes
- Schedule Later: False

Battery Check
…
Priority:
Period:
Schedule Later: True

Transition
…
Priority:
Period:
Schedule Later: True

Deploy State Tasks

The *Deploy* state

Initialize SD Card:
Write to the Deploy flag indicating
Priority:
Period: 30 minute
Schedule Later: False

Burn Wire Activate
…
Priority:
Period:
Schedule Later: True

Battery Check
…
Priority:
Period:
Schedule Later: True

Beacon
…
Priority:
Period:
Schedule Later: True

Transition
…
Priority:
Period:
Schedule Later: True

Science State Tasks

The *Science* state
Initialize Payload:
Write to the Deploy flag indicating
Priority:
Period: 30 minutes
Schedule Later: False

Receive Data
…
Priority:
Period:
Schedule Later: True

Record to Memory
…
Priority:
Period:
Schedule Later: True

Transition
…

Priority:
Period:
Schedule Later: True

Idle State Tasks

The *Idle* state

Comms State Tasks

The *Comms* state
Certain frequency for our own
We had one for IRU

Sean cooman
Protocals
Pyaload data
See if experiment went right
Battery data
Temperature
Time since deployment
IRE standard a command to the satellite that will stop the satellite from giving or recieivn data until we send a new cokmad

Command to instantly switch to certain states

FSM class

Within FSM it does all these?

Variables:

- Mission time (float?) -> net seconds
- power(float?)
    - high_power threshold (also a float)
    - low_power threshold (also float)
- Check that equipment is functional -> boolean
    - Subparts are functions
- Boolean sig -> (true if experiment started, false if experiment isn't started)
- Period variable (float)
    - sd_card_p
    - burn_wire_p
    - Battery_check_p
    - Beacon_p
    - transition_p
- Schedule Later variables
    - Boolean sd_card_SL
    - Boolean burn_wire_SL
    - Boolean battery_check_SL
    - Boolean beacon_SL
    - Boolean transition_SL
- Boolean solar_panels

Step by step upon launch:

- 30 minutes after ____(launch? stable orbit? System start/boot?) deploy antennas

In general

- Battery heaters and battery sensors constantly running in background

Method for deployment state (30 min before deploying) :

- Burn wire deployment - ???? idk about ts
- Initialize mission time var = 0 (to start it)
- sd_card_SL is False -> sd card needs to be initialized right now
- If power<=low_power_threshold -> go to idle state
- Else: proceed
- Check that equipment is functional - this should check several different parts -> will return (A & B & C & D & etc.) should return True
- Send handshake signal (should return some sort of data) back -> to mark that experiment stuff will start now

Method for science:

- Send some other sort of handshake signal (return data) -> to mark start of experiment
- Record

Method for text comms:
- Check if there are some certain requirements?
    - Turn on solar panels
- check between high and low power based on above variable of high/low power threshold

Method for idle state (which is basically when it's recharging):
- Automatic state upon reboot OR when cubesat power is low (power<low_power)

Method for recharging (input = current battery level)
- Charge until current level > low battery threshold

Steps that the thingie will go thru and conditionals

1. Upon turning the thing on manually (so something gets switched from false to true) deployment mode activates as the first mode
    a. Immediately starts the 30 min timer for the burn wire
    b. Initializes sd card
    c. Run method to check power level2
        i. If not high enough, send to idle (recharge) state
    d. Check functional equipment (how does this get checked thru code??)

    *btw steps a b c all happen simultanxeously
2. Transitional logic check will be IF b AND c AND d are all good, then proceed to the science and text comms stages
    a. If not, send to idle (recharge) state as any error in the power level or malfunctioning equipment is ass anyways

```
"""
CubeSat Finite State Machine
============================
States: DEPLOY, SCIENCE, COMMS, IDLE
Supporting: TaskScheduler, Task, hardware stubs
"""

import time
import random
from enum import Enum, auto
from dataclasses import dataclass, field
from typing import Callable, Optional
```

```
# ---------------------------------------------------------------------------
# Enums
# ---------------------------------------------------------------------------

class State(Enum):
    DEPLOY  = auto()
    SCIENCE = auto()
    COMMS   = auto()
    IDLE    = auto()


# ---------------------------------------------------------------------------
# Task
# ---------------------------------------------------------------------------

@dataclass
class Task:
    """
    A single schedulable unit of work.

    Attributes
    ----------
    name          : Human-readable label.
    priority      : Lower number = higher priority (1 is highest).
    period        : How often the task should run, in seconds.
                    Use None for one-shot tasks (run once, then remove).
    schedule_later: If True the scheduler may defer this task;
                    if False it must run at its next scheduled time.
    action        : The callable that performs the actual work.
    """
    name          : str
    priority      : int
    period        : Optional[float]         # None → one-shot
    schedule_later : bool
    action        : Callable[[], None]
    _next_run     : float = field(default=0.0, init=False, repr=False)
    _done         : bool  = field(default=False, init=False, repr=False)

    def is_ready(self, now: float) -> bool:
        return (not self._done) and (now >= self._next_run)

    def run(self, now: float) -> None:
        print(f"  [TASK] {self.name}")
        self.action()
```

```
        if self.period is None:          # one-shot → mark finished
            self._done = True
        else:
            self._next_run = now + self.period


# ---------------------------------------------------------------------------
# Task Scheduler
# ---------------------------------------------------------------------------

class TaskScheduler:
    """
    Runs ready tasks in priority order.
    One-shot tasks are automatically removed after execution.
    """

    def __init__(self) -> None:
        self._tasks: list[Task] = []

    def add(self, task: Task) -> None:
        self._tasks.append(task)
        self._tasks.sort(key=lambda t: t.priority)

    def clear(self) -> None:
        self._tasks.clear()

    def tick(self, now: float) -> None:
        """Call once per control loop iteration."""
        for task in list(self._tasks):
            if task._done:
                self._tasks.remove(task)
                continue
            if task.is_ready(now):
                task.run(now)


# ---------------------------------------------------------------------------
# Hardware Stubs
# ---------------------------------------------------------------------------
# Replace these with real hardware calls (GPIO, I2C, SPI, etc.)

class IMU:
    def self_test(self) -> bool:
        print("    [HW] IMU self-test … OK")
```

```
        return True

    def read(self) -> dict:
        return {"ax": 0.01, "ay": -0.02, "az": 9.81,
                "gx": 0.0,  "gy": 0.0,  "gz": 0.0}


class SDCard:
    def __init__(self) -> None:
        self._initialized = False

    def ping(self) -> bool:
        return self._initialized

    def initialize(self) -> bool:
        print("    [HW] SD card initializing …")
        self._initialized = True
        return True

    def write(self, data: dict) -> None:
        print(f"    [HW] SD write → {data}")


class Radio:
    def ping(self) -> bool:
        print("    [HW] Radio ping … OK")
        return True

    def send(self, payload: dict) -> bool:
        print(f"    [HW] Radio TX → {payload}")
        return True


class BurnWire:
    def ready(self) -> bool:
        return True

    def activate(self, duration_sec: float = 2.0) -> None:
        print(f"    [HW] Burn-wire firing for {duration_sec}s …")
        time.sleep(duration_sec)   # replace with non-blocking timer on-board
        print("    [HW] Burn-wire complete")


class SolarPanels:
```

```
    def __init__(self) -> None:
        self.on = False

    def enable(self) -> None:
        self.on = True
        print("    [HW] Solar panels enabled")

    def disable(self) -> None:
        self.on = False
        print("    [HW] Solar panels disabled")


class PowerSystem:
    """Stub – replace with ADC reads from battery monitor IC."""

    HIGH_THRESHOLD = 80.0   # %
    LOW_THRESHOLD  = 20.0   # %

    def __init__(self) -> None:
        self._level = 95.0  # start fully charged

    @property
    def level(self) -> float:
        return self._level

    def is_high(self) -> bool:
        return self._level >= self.HIGH_THRESHOLD

    def is_low(self) -> bool:
        return self._level <= self.LOW_THRESHOLD

    def simulate_drain(self, amount: float = 1.0) -> None:
        """For testing only."""
        self._level = max(0.0, self._level - amount)

    def simulate_charge(self, amount: float = 5.0) -> None:
        """For testing only."""
        self._level = min(100.0, self._level + amount)


# ---------------------------------------------------------------------------
# CubeSat FSM
# ---------------------------------------------------------------------------
```

```
class CubeSatFSM:
    """
    Finite State Machine for a CubeSat mission.

    States
    ------
    DEPLOY  – Initial state after power-on.  Sets up hardware, waits 30 min,
              fires burn wire to deploy antennas, then checks health.
    SCIENCE – Runs IMU experiments and logs data to SD card.
    COMMS   – Turns on solar panels, transmits stored data packets.
    IDLE    – Low-power holding state; beacons periodically, waits for charge.

    Transitions
    -----------
    DEPLOY  → SCIENCE / COMMS   (after health checks pass)
    DEPLOY  → IDLE              (power too low or equipment fault)
    SCIENCE → RECHARGE (IDLE)   (power drops below low threshold)
    COMMS   → RECHARGE (IDLE)   (power drops below low threshold)
    IDLE    → TRANSITION LOGIC  (power recovers above high threshold)
    SCIENCE ↔ COMMS             (based on mission time weighting)
    """

    BURN_WIRE_DELAY   = 30 * 60   # 30 minutes in seconds
    SCIENCE_WEIGHT_MAX = 1.0      # 100 % science at t=0
    SCIENCE_WEIGHT_MIN = 0.1      # 10  % science at end-of-life
    MISSION_EOL        = 90 * 24 * 3600  # 90-day mission

    def __init__(self) -> None:
        # Hardware
        self.imu         = IMU()
        self.sd          = SDCard()
        self.radio       = Radio()
        self.burn_wire   = BurnWire()
        self.solar_panels = SolarPanels()
        self.power       = PowerSystem()

        # Mission state
        self.state           : State = State.DEPLOY
        self.mission_time    : float = 0.0   # seconds since boot
        self.experiment_active: bool  = False
        self.data_buffer     : list  = []    # packets waiting to be downlinked

        # Scheduler
        self.scheduler = TaskScheduler()
```

```
    # Enter the initial state
    self._enter_state(State.DEPLOY)

# ------------------------------------------------------------------
# Main loop
# ------------------------------------------------------------------

def run(self, iterations: int = 100, tick_sec: float = 1.0) -> None:
    """
    Drive the FSM.  In real flight software this would be an infinite loop
    on a dedicated thread/RTOS task.
    """
    for _ in range(iterations):
        now = self.mission_time
        print(f"\n—— t={now:.0f}s  state={self.state.name}  "
              f"power={self.power.level:.1f}% ——")

        # 1. Run any due tasks
        self.scheduler.tick(now)

        # 2. Check for forced transition to IDLE (low power guard)
        if self.state != State.IDLE and self.power.is_low():
            print("  [FSM] Power low → IDLE")
            self._enter_state(State.IDLE)

        # 3. Simulate time passing (remove in production)
        self.mission_time += tick_sec
        self.power.simulate_drain(0.5)   # remove in production

# ------------------------------------------------------------------
# State entry points
# ------------------------------------------------------------------

def _enter_state(self, new_state: State) -> None:
    print(f"  [FSM] → entering {new_state.name}")
    self.state = new_state
    self.scheduler.clear()

    if new_state == State.DEPLOY:
        self._setup_deploy_tasks()
    elif new_state == State.SCIENCE:
        self._setup_science_tasks()
    elif new_state == State.COMMS:
```

```
        self._setup_comms_tasks()
    elif new_state == State.IDLE:
        self._setup_idle_tasks()

# ------------------------------------------------------------------
# DEPLOY state tasks
# ------------------------------------------------------------------

def _setup_deploy_tasks(self) -> None:
    """
    Priority order:
      1  Burn wire (fires once after 30-min delay)
      2  Initialize SD card (one-shot, immediate)
      3  Battery check (periodic)
      4  Beacon (periodic)
      5  Transition check (periodic)
    """
    burn = Task(
        name="Burn Wire Activate",
        priority=1,
        period=None,              # one-shot
        schedule_later=False,
        action=self._action_burn_wire,
    )
    burn._next_run = self.mission_time + self.BURN_WIRE_DELAY

    sd_init = Task(
        name="Initialize SD Card",
        priority=2,
        period=None,              # one-shot
        schedule_later=False,
        action=self._action_init_sd_card,
    )

    battery = Task(
        name="Battery Check",
        priority=3,
        period=5 * 60,            # every 5 min
        schedule_later=True,
        action=self._action_battery_check,
    )

    beacon = Task(
        name="Beacon",
```

```
        priority=4,
        period=30 * 60,           # every 30 min
        schedule_later=True,
        action=self._action_beacon,
    )

    transition = Task(
        name="Transition Check",
        priority=5,
        period=60,                # every 60 s
        schedule_later=True,
        action=self._action_transition_check,
    )

    for t in (burn, sd_init, battery, beacon, transition):
        self.scheduler.add(t)

# ---------------------------------------------------------------------
# SCIENCE state tasks
# ---------------------------------------------------------------------

def _setup_science_tasks(self) -> None:
    """
    Priority order:
      1  Initialize payload (one-shot)
      2  Receive / record data (periodic)
      3  Record to memory (periodic)
      5  Transition check (periodic)
    """
    init_payload = Task(
        name="Initialize Payload",
        priority=1,
        period=None,
        schedule_later=False,
        action=self._action_init_payload,
    )

    receive = Task(
        name="Receive IMU Data",
        priority=2,
        period=10,                # sample every 10 s
        schedule_later=False,
        action=self._action_receive_data,
    )
```

```
    record = Task(
        name="Record to Memory",
        priority=3,
        period=30 * 60,            # flush to SD every 30 min
        schedule_later=True,
        action=self._action_record_to_memory,
    )

    transition = Task(
        name="Transition Check",
        priority=5,
        period=60,
        schedule_later=True,
        action=self._action_transition_check,
    )

    for t in (init_payload, receive, record, transition):
        self.scheduler.add(t)

# ---------------------------------------------------------------------
# COMMS state tasks
# ---------------------------------------------------------------------

def _setup_comms_tasks(self) -> None:
    """
    Priority order:
      1  Enable solar panels (one-shot)
      2  Battery check (periodic)
      3  Send data packets (periodic, while in ground-station window)
      5  Transition check (periodic)
    """
    solar = Task(
        name="Enable Solar Panels",
        priority=1,
        period=None,
        schedule_later=False,
        action=self._action_enable_solar,
    )

    battery = Task(
        name="Battery Check",
        priority=2,
        period=5 * 60,
```

```
        schedule_later=True,
        action=self._action_battery_check,
    )

    send = Task(
        name="Send Data Packets",
        priority=3,
        period=60,              # attempt downlink every 60 s
        schedule_later=True,
        action=self._action_send_data_packets,
    )

    transition = Task(
        name="Transition Check",
        priority=5,
        period=60,
        schedule_later=True,
        action=self._action_transition_check,
    )

    for t in (solar, battery, send, transition):
        self.scheduler.add(t)

# ---------------------------------------------------------------
# IDLE state tasks
# ---------------------------------------------------------------

def _setup_idle_tasks(self) -> None:
    """
    Minimal tasks – conserve power.
      1  Beacon (every 30 min)
      2  Battery check (every 5 min) → exit IDLE when power recovers
    """
    beacon = Task(
        name="Beacon",
        priority=1,
        period=30 * 60,
        schedule_later=True,
        action=self._action_beacon,
    )

    battery = Task(
        name="Battery Check (Idle)",
        priority=2,
```

```
        period=5 * 60,
        schedule_later=True,
        action=self._action_idle_battery_check,
    )

    for t in (beacon, battery):
        self.scheduler.add(t)

# ---------------------------------------------------------------
# Action implementations
# ---------------------------------------------------------------

def _action_burn_wire(self) -> None:
    """Fire the burn wire to deploy antennas."""
    self.burn_wire.activate(duration_sec=2.0)

def _action_init_sd_card(self) -> None:
    """Initialize SD card and write deploy flag."""
    success = self.sd.initialize()
    if success:
        self.sd.write({"flag": "DEPLOYED", "t": self.mission_time})

def _action_battery_check(self) -> None:
    """Log battery level.  Low-power guard in run() handles transitions."""
    print(f"    [POWER] Battery at {self.power.level:.1f}%")

def _action_beacon(self) -> None:
    """Transmit a minimal status beacon."""
    self.radio.send({
        "type"        : "BEACON",
        "state"       : self.state.name,
        "mission_time": self.mission_time,
        "power"       : self.power.level,
    })

def _action_transition_check(self) -> None:
    """
    Transition Logic diamond from the FSM diagram.

    Conditions to leave DEPLOY:
      - SD card initialized
      - Equipment functional
      - Power above low threshold
```

```
    Once healthy, decide SCIENCE vs COMMS by mission-time weighting.
    """
    if self.state == State.DEPLOY:
        if not self.sd.ping():
            print("    [TRANSITION] SD not ready – staying in DEPLOY")
            return
        if not self._check_equipment_functional():
            print("    [TRANSITION] Equipment fault – → IDLE")
            self._enter_state(State.IDLE)
            return
        if self.power.is_low():
            print("    [TRANSITION] Power low – → IDLE")
            self._enter_state(State.IDLE)
            return
        # All checks passed – send handshake and move on
        self._action_send_handshake()
        self._enter_state(self._choose_science_or_comms())
        return

    if self.state in (State.SCIENCE, State.COMMS):
        # Periodic re-evaluation: shift weight toward COMMS over time
        next_state = self._choose_science_or_comms()
        if next_state != self.state:
            print(f"    [TRANSITION] Switching → {next_state.name}")
            self._enter_state(next_state)

def _action_init_payload(self) -> None:
    """Signal start of experiment and run IMU self-test."""
    ok = self.imu.self_test()
    self.experiment_active = ok
    self.radio.send({"type": "EXPERIMENT_START", "t": self.mission_time})
    print(f"    [SCIENCE] Experiment active: {ok}")

def _action_receive_data(self) -> None:
    """Read IMU and buffer a timestamped sample."""
    if not self.experiment_active:
        return
    reading = self.imu.read()
    packet = {"t": self.mission_time, "imu": reading}
    self.data_buffer.append(packet)
    print(f"    [SCIENCE] Buffered sample #{len(self.data_buffer)}")

def _action_record_to_memory(self) -> None:
    """Flush buffered samples to SD card."""
```

```
    if not self.data_buffer:
        return
    for packet in self.data_buffer:
        self.sd.write(packet)
    print(f"    [SCIENCE] Flushed {len(self.data_buffer)} packets to SD")
    self.data_buffer.clear()

def _action_enable_solar(self) -> None:
    self.solar_panels.enable()

def _action_send_data_packets(self) -> None:
    """
    Randomly select and downlink a stored data packet.
    (Mirrors 'Randomly select a data packet' node in diagram.)
    In production: check ground-station contact window first.
    """
    if not self.data_buffer:
        print("    [COMMS] No packets to send")
        return
    packet = random.choice(self.data_buffer)
    success = self.radio.send({"type": "DATA", "payload": packet})
    if success:
        self.data_buffer.remove(packet)

def _action_idle_battery_check(self) -> None:
    """Exit IDLE once battery has recovered above HIGH threshold."""
    print(f"    [IDLE] Battery at {self.power.level:.1f}%")
    if self.power.is_high():
        print("    [IDLE] Power recovered → Transition Logic")
        self._enter_state(self._choose_science_or_comms())

def _action_send_handshake(self) -> None:
    """Handshake signal to ground to confirm deploy health."""
    self.radio.send({
        "type"   : "HANDSHAKE",
        "status" : "DEPLOY_COMPLETE",
        "power"  : self.power.level,
        "t"      : self.mission_time,
    })

# ---------------------------------------------------------------
# Helpers
# ---------------------------------------------------------------
```

```
    def _check_equipment_functional(self) -> bool:
        """
        Return True only if every subsystem passes its self-test.
        Short-circuit evaluation – stops at first failure.
        """
        return (
            self.imu.self_test()   and
            self.sd.ping()         and
            self.radio.ping()      and
            self.burn_wire.ready()
        )

    def _choose_science_or_comms(self) -> State:
        """
        'Decrease Science and Increase Comms over Time' decision diamond.

        Science weight starts at SCIENCE_WEIGHT_MAX (1.0) and linearly
        decreases to SCIENCE_WEIGHT_MIN (0.1) by end-of-life.
        A random draw against that weight picks the next state.
        """
        t = min(self.mission_time, self.MISSION_EOL)
        science_weight = (
            self.SCIENCE_WEIGHT_MAX
            - (self.SCIENCE_WEIGHT_MAX - self.SCIENCE_WEIGHT_MIN)
            * (t / self.MISSION_EOL)
        )
        chosen = State.SCIENCE if random.random() < science_weight else State.COMMS
        print(f"    [TRANSITION] science_weight={science_weight:.2f} → {chosen.name}")
        return chosen


# ---------------------------------------------------------------------------
# Entry point (smoke test)
# ---------------------------------------------------------------------------

if __name__ == "__main__":
    sat = CubeSatFSM()

    # Fast-forward past the 30-min burn-wire delay for demo purposes
    sat.scheduler._tasks[0]._next_run = 5   # fire burn wire at t=5s

    # Run 20 ticks at 1 s each (real flight: loop indefinitely)
    sat.run(iterations=20, tick_sec=1.0)
```

```
self._tasks: dict = {
            State.IDLE:    self._build_idle_tasks(),
            State.DEPLOY:  self._build_deploy_tasks(),
            State.SCIENCE: self._build_science_tasks(),
            State.COMMS:   self._build_comms_tasks(),
        }
```

Cd into folder
Git clone http:link
Make a new branch
Make changes + commit them
Check out to main
Branch -> Merge with the new branch (ts will push it)
Make sure it’s all updated
Delete the branch

```
"""
CubeSat Finite State Machine (FSM)
==================================
States: IDLE -> DEPLOY -> SCIENCE -> COMMS -> (back to IDLE as needed)
"""

import time
from enum import Enum, auto


# ---------------------------------------------------------------------------
# Enums
# ---------------------------------------------------------------------------

class State(Enum):
    IDLE    = auto()
    DEPLOY  = auto()
    SCIENCE = auto()
    COMMS   = auto()


class TaskResult(Enum):
    SUCCESS = auto()
    FAILURE = auto()
    SKIPPED = auto()


# ---------------------------------------------------------------------------
# Task descriptor  (mirrors your "Priority / Period / Schedule Later" spec)
# ---------------------------------------------------------------------------

class Task:
    """
    Lightweight wrapper that holds metadata about a recurring FSM task.

    Parameters
    ----------
    name          : human-readable label
    priority      : lower number = higher priority (1 is highest)
    period_s      : how often the task should run, in seconds
    schedule_later: if True the task is queued for later; if False it runs immediately
    func          : callable implementing the task logic, signature () -> TaskResult
    """
```

```
    def __init__(self, name: str, priority: int, period_s: float,
                 schedule_later: bool, func):
        self.name           = name
        self.priority       = priority
        self.period_s       = period_s
        self.schedule_later = schedule_later
        self.func           = func
        self.last_run_time  = None          # wall-clock seconds (time.monotonic)

    def is_due(self, now: float) -> bool:
        if self.last_run_time is None:
            return True
        return (now - self.last_run_time) >= self.period_s

    def run(self) -> TaskResult:
        result = self.func()
        self.last_run_time = time.monotonic()
        return result

    def __repr__(self):
        return (f"Task({self.name!r}, priority={self.priority}, "
                f"period={self.period_s}s, schedule_later={self.schedule_later})")


# ---------------------------------------------------------------------------
# CubeSat FSM
# ---------------------------------------------------------------------------

class CubeSatFSM:
    """
    Finite State Machine for CubeSat mission management.

    Key design decisions
    --------------------
    * Power is represented as a float in [0.0, 1.0] (fraction of full charge).
    * Mission time counts seconds since the FSM was started (i.e. since boot).
    * Each state has its own ordered task list; tasks are sorted by priority
      so that if time is short the most critical work happens first.
    * schedule_later=False tasks run immediately on state entry;
      schedule_later=True tasks are deferred until their period has elapsed.
    """

    # -----------------------------------------------------------------
    # Thresholds
```

```
# ---------------------------------------------------------------
LOW_POWER_THRESHOLD  = 0.20   # drop below -> forced IDLE
HIGH_POWER_THRESHOLD = 0.80   # rise above -> allowed to leave IDLE

# Deployment window: 30 minutes after boot
DEPLOY_DELAY_S = 30 * 60

def __init__(self, initial_power: float = 1.0):
    # Core state
    self.state: State             = State.IDLE
    self.mission_time: float      = 0.0          # seconds since boot
    self._boot_time: float        = time.monotonic()
    self._state_entry_time: float  = time.monotonic()

    # Power model
    self.power: float             = initial_power

    # Mission flags
    self.experiment_started: bool  = False    # True once science begun
    self.antennas_deployed: bool   = False
    self.sd_card_initialized: bool = False
    self.deploy_flag_written: bool = False
    self.solar_panels_on: bool     = False

    # Equipment health  (True = functional)
    self.equipment_ok: dict = {
        "sd_card":    False,
        "burn_wire":  False,
        "radio":      False,
        "payload":    False,
        "solar_panel": False,
    }

    # Period configuration (seconds) — override before calling run() if needed
    self.periods = {
        "beacon":        30 * 60,
        "battery_check": 5  * 60,
        "transition":    1  * 60,
        "sd_card":       30 * 60,
        "burn_wire":     30 * 60,
        "receive_data":  10 * 60,
        "record_memory": 10 * 60,
        "init_payload":  30 * 60,
    }
```

```
    # Build task lists for each state
    self._tasks: dict = {
        State.IDLE:    self._build_idle_tasks(),
        State.DEPLOY:  self._build_deploy_tasks(),
        State.SCIENCE: self._build_science_tasks(),
        State.COMMS:   self._build_comms_tasks(),
    }

# ======================================================================
# Public entry point
# ======================================================================

def run(self):
    """Main loop — call once per iteration of your scheduler."""
    self.mission_time = time.monotonic() - self._boot_time

    # Global safety check: power too low -> force IDLE
    if self.power <= self.LOW_POWER_THRESHOLD and self.state != State.IDLE:
        print(f"[FSM] Power low ({self.power:.0%}) — forcing IDLE")
        self._transition_to(State.IDLE)
        return

    # Dispatch to current state handler
    handler = {
        State.IDLE:    self._run_idle,
        State.DEPLOY:  self._run_deploy,
        State.SCIENCE: self._run_science,
        State.COMMS:   self._run_comms,
    }[self.state]

    handler()

# ======================================================================
# State handlers
# ======================================================================

def _run_idle(self):
    """
    IDLE — entered on reboot or when power is critically low.
    Minimal activity: only beacon + battery monitoring.
    Transitions to DEPLOY once power is sufficient and deploy window opens.
    """
    self._execute_due_tasks(State.IDLE)
```

```
        # Recharge until power rises above HIGH threshold
        if self.power < self.HIGH_POWER_THRESHOLD:
            self._recharge(target=self.HIGH_POWER_THRESHOLD)
            return  # stay in IDLE while charging

        # Decide next state once power is sufficient
        if not self.antennas_deployed and self.mission_time >= self.DEPLOY_DELAY_S:
            self._transition_to(State.DEPLOY)
        elif self.antennas_deployed and not self.experiment_started:
            self._transition_to(State.SCIENCE)
        elif self.experiment_started:
            self._transition_to(State.COMMS)

    def _run_deploy(self):
        """
        DEPLOY — initialise SD card, fire burn wire, deploy antennas.
        Transitions to SCIENCE after antennas are confirmed deployed.
        """
        self._execute_due_tasks(State.DEPLOY)

        if self.antennas_deployed and self._check_equipment():
            self._transition_to(State.SCIENCE)

    def _run_science(self):
        """
        SCIENCE — initialise payload, collect and record experiment data.
        """
        self._execute_due_tasks(State.SCIENCE)

    def _run_comms(self):
        """
        COMMS — uplink/downlink window; solar panels enabled for extra power.
        """
        self._execute_due_tasks(State.COMMS)

    # ================================================================
    # Task builders
    # ================================================================

    def _build_idle_tasks(self) -> list:
        tasks = [
            Task("Beacon",          1, self.periods["beacon"],       schedule_later=False,
func=self._task_beacon),
```

```
            Task("Battery Check",    2, self.periods["battery_check"], schedule_later=True,
func=self._task_battery_check),
            Task("Transition Check", 3, self.periods["transition"],    schedule_later=True,
func=self._task_transition_check),
        ]
        return sorted(tasks, key=lambda t: t.priority)

    def _build_deploy_tasks(self) -> list:
        tasks = [
            Task("Initialize SD Card",   1, self.periods["sd_card"],       schedule_later=False,
func=self._task_init_sd_card),
            Task("Burn Wire Activate",   2, self.periods["burn_wire"],     schedule_later=True,
func=self._task_burn_wire),
            Task("Battery Check",        3, self.periods["battery_check"], schedule_later=True,
func=self._task_battery_check),
            Task("Beacon",               4, self.periods["beacon"],        schedule_later=True,
func=self._task_beacon),
            Task("Transition Check",     5, self.periods["transition"],    schedule_later=True,
func=self._task_transition_check),
        ]
        return sorted(tasks, key=lambda t: t.priority)

    def _build_science_tasks(self) -> list:
        tasks = [
            Task("Initialize Payload",  1, self.periods["init_payload"],   schedule_later=False,
func=self._task_init_payload),
            Task("Receive Data",        2, self.periods["receive_data"],   schedule_later=True,
func=self._task_receive_data),
            Task("Record to Memory",    3, self.periods["record_memory"],  schedule_later=True,
func=self._task_record_memory),
            Task("Transition Check",    4, self.periods["transition"],     schedule_later=True,
func=self._task_transition_check),
        ]
        return sorted(tasks, key=lambda t: t.priority)

    def _build_comms_tasks(self) -> list:
        tasks = [
            Task("Solar Panel Enable",  1, self.periods["battery_check"], schedule_later=False,
func=self._task_enable_solar_panels),
            Task("Battery Check",       2, self.periods["battery_check"], schedule_later=True,
func=self._task_battery_check),
            Task("Beacon",              3, self.periods["beacon"],        schedule_later=True,
func=self._task_beacon),
```

```
        Task("Transition Check",    4, self.periods["transition"],     schedule_later=True,
func=self._task_transition_check),
        ]
        return sorted(tasks, key=lambda t: t.priority)

    # ================================================================
    # Task implementations  (stubs — replace with real hardware calls)
    # ================================================================

    def _task_beacon(self) -> TaskResult:
        """Transmit a beacon signal with current telemetry."""
        telemetry = {
            "state":        self.state.name,
            "mission_time": self.mission_time,
            "power":        self.power,
            "exp_started":  self.experiment_started,
        }
        print(f"[BEACON] Telemetry: {telemetry}")
        # TODO: call radio driver here
        return TaskResult.SUCCESS

    def _task_battery_check(self) -> TaskResult:
        """Read battery sensor and update self.power."""
        # TODO: read from ADC / battery sensor driver
        print(f"[BATTERY CHECK] Power level: {self.power:.1%}")
        if self.power <= self.LOW_POWER_THRESHOLD:
            print("[BATTERY CHECK] WARNING: Low power detected!")
        return TaskResult.SUCCESS

    def _task_transition_check(self) -> TaskResult:
        """Log current state for transition evaluation."""
        print(f"[TRANSITION CHECK] state={self.state.name}, "
              f"power={self.power:.1%}, mission_time={self.mission_time:.0f}s")
        return TaskResult.SUCCESS

    def _task_init_sd_card(self) -> TaskResult:
        """Initialize the SD card and write the deploy flag."""
        print("[SD CARD] Initializing SD card ...")
        # TODO: call SD card driver / SPI init
        self.sd_card_initialized     = True
        self.equipment_ok["sd_card"] = True

        # Write deploy flag to SD
        self.deploy_flag_written = True
```

```
        print("[SD CARD] Deploy flag written")

        # Zero mission timer at deploy-state entry
        self.mission_time = 0.0
        self._boot_time   = time.monotonic()

        # Send handshake to ground / payload to signal deploy readiness
        self._send_handshake("DEPLOY_READY")
        return TaskResult.SUCCESS

    def _task_burn_wire(self) -> TaskResult:
        """Fire burn wire to deploy antennas."""
        if not self.sd_card_initialized:
            print("[BURN WIRE] SD card not ready — skipping")
            return TaskResult.SKIPPED

        print("[BURN WIRE] Activating burn wire ...")
        # TODO: actuate GPIO pin for burn wire
        self.antennas_deployed          = True
        self.equipment_ok["burn_wire"]  = True
        print("[BURN WIRE] Antennas deployed")
        return TaskResult.SUCCESS

    def _task_init_payload(self) -> TaskResult:
        """Initialize the science payload and send start handshake."""
        print("[PAYLOAD] Initializing payload ...")
        # TODO: bring up payload hardware
        self.equipment_ok["payload"] = True
        self.experiment_started      = True
        self._send_handshake("SCIENCE_START")
        print("[PAYLOAD] Payload ready, experiment started")
        return TaskResult.SUCCESS

    def _task_receive_data(self) -> TaskResult:
        """Receive data from the science payload."""
        if not self.equipment_ok["payload"]:
            print("[RCV DATA] Payload not ready")
            return TaskResult.FAILURE
        print("[RCV DATA] Receiving payload data ...")
        # TODO: read from payload interface
        return TaskResult.SUCCESS

    def _task_record_memory(self) -> TaskResult:
        """Write received data to SD card."""
```

```
    if not self.sd_card_initialized:
        print("[RECORD] SD card not initialized")
        return TaskResult.FAILURE
    print("[RECORD] Writing data to SD card ...")
    # TODO: write buffer to SD card
    return TaskResult.SUCCESS

def _task_enable_solar_panels(self) -> TaskResult:
    """Enable solar panels for extra power during comms window."""
    print("[SOLAR] Enabling solar panels ...")
    # TODO: toggle solar panel GPIO / power switch
    self.solar_panels_on            = True
    self.equipment_ok["solar_panel"] = True
    return TaskResult.SUCCESS

# ======================================================================
# Helpers
# ======================================================================

def _execute_due_tasks(self, state: State):
    """
    Run all tasks for `state` that are currently due.

    schedule_later=False  -> run immediately on state entry (last_run_time is None),
                             then follow normal period cadence.
    schedule_later=True   -> skip on first call; only run once period has elapsed.
    """
    now = time.monotonic()
    for task in self._tasks[state]:
        first_call = task.last_run_time is None

        if first_call and task.schedule_later:
            task.last_run_time = now   # mark as "seen" but don't run yet
            print(f"[FSM] Deferring '{task.name}' (schedule_later=True)")
            continue

        if task.is_due(now):
            print(f"[FSM] Running '{task.name}' (priority={task.priority})")
            result = task.run()
            print(f"[FSM]   -> Result: {result.name}")

def _transition_to(self, new_state: State):
    if new_state == self.state:
        return
```

```
        print(f"\n[FSM] Transition: {self.state.name} -> {new_state.name}\n")
        self.state            = new_state
        self._state_entry_time = time.monotonic()
        # Reset task timers so immediate tasks fire fresh on state entry
        for task in self._tasks[new_state]:
            task.last_run_time = None

    def _recharge(self, target: float):
        """
        Wait for battery to reach target level.
        In real firmware this returns immediately and is polled each scheduler cycle.
        """
        print(f"[RECHARGE] Charging... (current: {self.power:.1%}, target: {target:.1%})")
        # TODO: in real code, just return here. The FSM will call run() again next tick.

    def _check_equipment(self) -> bool:
        """Return True only if all equipment reports healthy."""
        ok = all(self.equipment_ok.values())
        if not ok:
            faulty = [k for k, v in self.equipment_ok.items() if not v]
            print(f"[EQUIPMENT] Faulty subsystems: {faulty}")
        return ok

    def _send_handshake(self, signal: str) -> dict:
        """Transmit a named handshake signal; return acknowledgement data."""
        print(f"[HANDSHAKE] Sending '{signal}' ...")
        # TODO: call radio driver; parse and return real ACK
        ack = {"signal": signal, "ack": True, "mission_time": self.mission_time}
        print(f"[HANDSHAKE] ACK: {ack}")
        return ack


# ---------------------------------------------------------------------------
# Quick smoke-test
# ---------------------------------------------------------------------------

if __name__ == "__main__":
    fsm = CubeSatFSM(initial_power=0.9)

    print("=== CubeSat FSM Smoke Test ===\n")
    for tick in range(5):
        print(f"--- Tick {tick + 1} ---")
        fsm.run()
        print()
```

```
time.sleep(0.1)
```

Checking battery website (from pycubed):
https://pycubed.org/pycubed%20py%20b80aabda74284488bfd9b8ef46a0eba2.html